`redist-ensemble`: A High-Performance Rust Implementation of ReCom for Redistricting Feasibility Analysis

H.2 — Rust Ensemble Engine

Gio Della-Libera

Apportionment Research Group

giodl@microsoft.com

May 2026

Abstract

Markov chain ensemble analysis has become a primary tool for detecting partisan gerrymandering in state-court litigation following *Rucho v. Common Cause* (2019). The dominant implementation, GerryChain (Python), achieves approximately 21–36 steps per second for congressional-scale states (North Carolina: 2,672 tracts, $k = 14$), making a 10,000-step ensemble a 5–8 minute computation. This throughput limits ensemble analysis to an offline, post-hoc step run separately from the redistricting pipeline.

We present `redist-ensemble`, a Rust implementation of the ReCom Markov chain using Wilson's uniform random spanning tree algorithm [Wilson, 1996]. Wilson's algorithm runs in expected time equal to the cover time of a random walk on the subgraph; for planar graphs this is $O(|V| \log |V|)$ [Aldous, 1990], reducing the per-step cost from Python's interpreted execution to native compiled arithmetic at roughly 50,000 steps per second—an estimated 2,300× speedup over GerryChain. At this throughput, a 10,000-step ensemble for North Carolina completes in approximately 0.2

seconds, enabling ensemble analysis as an integrated real-time audit step within the `redist build` pipeline rather than an overnight computation.

The implementation consists of three modules: `spanning.rs` (Wilson's loop-erased random walk), `recom.rs` (the ReCom proposal with correct stationary-distribution balance-cut enumeration), and `chain.rs` (parallel runner with Rayon, per-chain SHA-256 seeds, $\hat{R}$, ESS, and Hamming autocorrelation diagnostics). Benchmark validation against `criterion.rs` is planned for Phase 2.

Contents

1. Introduction

1.1. The Ensemble Bottleneck

Following *Rucho v. Common Cause* [Supreme Court of the United States, 2019], partisan gerrymandering claims have migrated from federal to state courts, and Markov chain ensemble analysis has become the principal expert-witness tool for showing that an enacted map is a statistical outlier. The dominant workflow runs thousands of algorithmically drawn plans using the ReCom chain [DeFord et al., 2021], then places the contested map within the resulting distribution. A plan at the 99th percentile of Republican seat share is characterized as strong evidence of partisan intent; at the 50th percentile, it is indistinguishable from a random draw.

The limiting factor is throughput. The primary open-source implementation, GerryChain [MGGG Redistricting Lab, 2021], is written in Python. For North Carolina (2,672 census tracts, $k = 14$ districts), GerryChain achieves approximately 21 steps per second. A credible 10,000-step ensemble therefore requires roughly 8 minutes; a 50,000-step ensemble requires 40 minutes. At this speed, ensemble analysis is necessarily a separate, overnight computation—a post-hoc audit distinct from the redistricting pipeline itself.

This throughput ceiling creates two practical problems. First, ensemble analysis cannot serve as a *real-time* feasibility check during map-drawing. A redistricting practitioner cannot interactively explore whether a proposed boundary change moves the plan toward or away from the ensemble median. Second, the computational cost discourages the thorough convergence diagnostics—multiple parallel chains, extended runs—that would give courts and litigants justified confidence in the ensemble's stationarity (see Della-Libera 2026d).

1.2. The Rust Solution

We present redist-ensemble, a Rust implementation of the ReCom Markov chain built around Wilson's uniform random spanning tree algorithm [Wilson, 1996]. The design goal is to collapse the throughput gap sufficiently that a 10,000-step ensemble for any congressional-scale state completes in under one second, enabling ensemble analysis to run as an integrated step within `redist build` rather than as a separate overnight job.

The key algorithmic innovation is the use of Wilson's loop-erased random walk [Lawler,

1980] to generate uniform random spanning trees. Wilson's algorithm runs in $O(\tau_{\text{cover}})$ expected time [Wilson, 1996], where τ_{cover} is the cover time of the underlying random walk. For planar graphs—the natural setting for census-tract adjacency graphs—Aldous [1990] established that $\tau_{\text{cover}} = O(|V| \log |V|)$; the planar bound is his result, not Wilson's. Each ReCom step uses the spanning tree of the merged pair region (approximately $2n/k$ nodes, where n is the total tract count and k is the number of districts), so Wilson's cost per step is $O((n/k) \log(n/k))$. Rust's compiled, zero-overhead arithmetic exploits this theoretical efficiency to an extent that interpreted Python cannot.

1.3. Pipeline Integration

The intended deployment is as the `redist ensemble` subcommand, integrated into the same binary as `redist state`, `redist analyze`, and `redist map`. A typical invocation:

```
redist ensemble --state NC --year 2020 --steps 10000 \
                --chains 8 --seed 42 --out nc_ensemble.json
```

At 50,000 steps/sec, this completes in approximately 0.2 seconds for North Carolina. The output JSON carries $\hat{R}$, ESS, and Hamming autocorrelation diagnostics, enabling immediate convergence assessment without a separate analysis pass.

1.4. Paper Organization

Section 2 reviews ReCom, Wilson's spanning tree algorithm, and GerryChain. Section 3 gives the formal algorithmic description. Section 4 describes the Rust implementation architecture. Section 5 presents the performance comparison. Section 6 describes the convergence diagnostics. Section 7 concludes.

2. Related Work

2.1. ReCom and GerryChain

The ReCom (*Recombination*) Markov chain was introduced by DeFord et al. [2021] as a method for sampling redistricting plans with a provably well-defined stationary distribution. At each step, the chain selects a pair of adjacent districts, merges them into a single region,

generates a random spanning tree of the merged region using Wilson's algorithm, finds all balanced bipartitions of that tree, and selects one uniformly at random. The chain's stationary distribution is uniform over the space of population-balanced redistricting plans, up to the spanning-tree bias introduced by graph structure.

GerryChain [MGGG Redistricting Lab, 2021] is the canonical open-source Python implementation of ReCom, developed and maintained by the MGGG Redistricting Lab at Tufts University. It has been used in expert reports submitted in redistricting litigation in North Carolina [Herschlag et al., 2020], Wisconsin, and Pennsylvania [Chikina et al., 2017], and forms the computational backbone of the ensemble-based auditing literature. GerryChain is designed for correctness and research flexibility over raw throughput; it runs at approximately 21–36 steps per second for congressional-scale states.

Autry et al. [2021] extended the ReCom framework to a multiscale forest variant, improving mixing properties for large k. Carter et al. [2019] analyzed the *merge-split* chain, a closely related Markov chain with distinct stationary distribution properties.

2.2. Wilson's Spanning Tree Algorithm

Wilson [1996] gave the first efficient algorithm for generating uniform random spanning trees of an arbitrary graph. Wilson's algorithm performs a loop-erased random walk from an arbitrary vertex to the already-constructed tree, erasing loops as they form and adding the loop-erased path to the tree. The algorithm runs in expected time equal to the *mean cover time* of a simple random walk on the graph.

The loop-erasing property ensures that the resulting spanning tree is drawn uniformly from the set of all spanning trees of the graph (by Kirchhoff's matrix-tree theorem), making it a direct tool for ReCom proposals. Aldous [1990] proved that the cover time of a random walk on a planar graph with m vertices is $O(m \log m)$, giving Wilson's algorithm $O(m \log m)$ expected time on census-tract adjacency subgraphs.

The loop-erased random walk has a long independent history dating to Lawler [1980], who studied it as a model in statistical mechanics. Propp and Wilson [1998] showed that Wilson's algorithm is a special case of their general *coupling from the past* framework.

2.3. Rust for High-Performance Scientific Computing

Rust's combination of zero-cost abstractions, safe concurrency, and predictable memory layout has made it an increasingly attractive language for computationally intensive scientific applications. The Rayon data-parallelism library [Rayon Contributors, 2024] provides fork-join parallelism with work-stealing scheduling and a threading model that maps cleanly onto independent MCMC chains. Matsakis and Klock [2014] describe the language's ownership system, which eliminates data races by construction—a critical property for parallel MCMC where chains share no mutable state.

Within the `redist` ecosystem, the Rust codebase has achieved approximately 213× speedup over the Python pipeline for the core METIS partitioning workflow [Della-Libera, 2026a]. The REDIST-ENSEMBLE crate extends this philosophy to ensemble sampling.

2.4. Portfolio Context

The G-track papers in this portfolio provide the ensemble methodology and comparison results that REDIST-ENSEMBLE operationalizes. Della-Libera [2026b] establishes the G-track framework for comparing deterministic redistricting algorithms to GERRYCHAIN ensembles. Della-Libera [2026c] provides direct percentile placement of the APPORTIONREGIONS plans within GERRYCHAIN ensembles for six states. Della-Libera [2026d] formalizes the $\hat{R}$, ESS, and Hamming autocorrelation diagnostics implemented in the `redist` binary. The H.1 paper [Della-Libera, 2026e] establishes the bisection-ensemble connection that motivates real-time ensemble integration. REDIST-ENSEMBLE closes the final methodological gap: G-track results previously depended on the Python GERRYCHAIN installation; REDIST-ENSEMBLE makes those results reproducible from the `redist` Rust binary alone.

3. Algorithm

3.1. Setup and Notation

Let $G = (V, E)$ be the census-tract adjacency graph for a state with $n = |V|$ tracts and $|E|$ shared boundaries. Each vertex $v \in V$ carries a population $p(v)$. A *k-partition* of G is a function $\sigma : V \to \{1, \ldots, k\}$ such that each part $V_i = \sigma^{-1}(i)$ induces a connected subgraph of

G. A partition is *population-balanced* if

$$\left| \frac{\sum_{v \in V_i} p(v)}{\bar{p}} - 1 \right| \leq \varepsilon \quad \forall i \in \{1, \ldots, k\},$$

where $\bar{p} = \frac{1}{k} \sum_{v \in V} p(v)$ is the target district population and $\varepsilon = 0.005$ is the statutory tolerance.

3.2. Wilson's Loop-Erased Random Walk

Definition 1 (Uniform Random Spanning Tree)**.** *A* uniform random spanning tree *(UST) of a connected graph H is a spanning tree of H drawn uniformly at random from the set of all spanning trees of H.*

Wilson's algorithm [Wilson, 1996] generates a UST of H as follows:

Algorithm 1 Wilson's UST Algorithm

Require: Connected graph $H = (V_H, E_H)$, arbitrary root $r \in V_H$
Ensure: Uniform random spanning tree T of H

```
T ← {r}
while T ≠ V_H do
    Pick any u ∈ V_H \ T
    Perform a simple random walk from u until hitting T, recording the path π
    Erase all loops from π to obtain the loop-erased path LE(π)
    Add all vertices and edges of LE(π) to T
end while
return T
```

Theorem 1 (Wilson 1996)**.** *Algorithm 1 outputs a uniform random spanning tree of H in expected time $O(\tau_{\text{cover}}(H))$, where $\tau_{\text{cover}}(H)$ is the cover time of a simple random walk on H. For planar graphs with m vertices, $\tau_{\text{cover}}(H) = O(m \log m)$.*

In our setting, H is the subgraph induced by the union $V_i \cup V_j$ of two adjacent districts chosen for recombination. This subgraph has approximately $m = 2n/k$ vertices, giving an expected Wilson cost of $O((n/k) \log(n/k))$ per ReCom step.

3.3. The ReCom Proposal

Algorithm 2 describes one step of the ReCom chain. The critical design decision is the treatment of tree bipartitions: we enumerate *all* balanced cuts of the spanning tree rather than

sampling a single cut. This enumeration is necessary to replicate GerryChain's stationary distribution, which places weight on each spanning tree proportional to the number of balanced bipartitions it admits [DeFord et al., 2021].

Algorithm 2 One ReCom Step

Require: Current partition σ, population tolerance ε
Ensure: Updated partition σ'

resample_count $\leftarrow 0$
repeat
  if resample_count ≥ 10 **then**
    Select a new pair (i, j) uniformly from all adjacent district pairs
    resample_count $\leftarrow 0$
  end if
  Select pair (i, j) uniformly from all adjacent district pairs
  $R \leftarrow V_i \cup V_j$ ▷ Merged region
  $H \leftarrow G[R]$ ▷ Induced subgraph
  $T \leftarrow \textsc{WilsonUST}(H)$ ▷ Algorithm 1
  $\mathcal{B} \leftarrow \{e \in T : \text{removing } e \text{ gives a balanced bipartition}\}$ ▷ Enumerate all balanced cuts
  resample_count $\leftarrow$ resample_count $+ 1$
until $\mathcal{B} \neq \emptyset$
Sample $e^* \sim \text{Uniform}(\mathcal{B})$
$\{A, B\} \leftarrow$ bipartition of T induced by removing e^*
Update σ' by assigning $V_i \leftarrow A$, $V_j \leftarrow B$
return σ'

Failure handling. When a sampled spanning tree T of the merged region R admits no balanced bipartition (i.e., $\mathcal{B} = \emptyset$), the algorithm resamples the entire spanning tree from scratch rather than attempting to repair the same tree. This tree-resample-on-failure policy is essential: retrying with the same tree would not sample a new point from the UST distribution and would break the stationary distribution guarantee. If 10 consecutive spanning tree samples for the same district pair all fail to yield a balanced bipartition, the pair is abandoned and a new adjacent pair (i, j) is selected. This pair-reselection mechanism is not a Rust-specific workaround; the same failure mode occurs in Python GerryChain (Section 5 discusses the Texas case in detail). Rust's advantage is simply speed: it exhausts the resample budget faster and moves on.

Balance cut enumeration. For a spanning tree T with $m-1$ edges, each edge removal creates a unique bipartition of the m vertices into two subtrees. Each subtree's population is the sum of $p(v)$ over its vertices. We scan all $m-1$ edges in a single linear pass, accumulating the subtree population on one side via DFS, and classify each edge as balanced or unbalanced. The cost of this enumeration is $O(m)$, subdominant to the $O(m \log m)$ Wilson cost.

3.4. Parallel Chains

Independent chains are embarrassingly parallel: each chain maintains its own partition state, its own random-number generator seeded deterministically from the chain index, and its own step history. No synchronization is required during sampling; chains communicate only at the diagnostics aggregation step.

Per-chain seeds are derived deterministically as

$$seed_i = \text{SHA256}(\text{“ENSEMBLE\_CHAIN\_”} \| i \| \text{“\_”} \| base\_seed),$$

where i is the zero-indexed chain index and *base_seed* is the user-supplied seed. This construction ensures reproducibility across runs and across different chain counts while maintaining independence between chains.

The encoding is specified precisely to guarantee cross-environment reproducibility. Integer i is encoded as a 64-bit little-endian unsigned integer (`i.to_le_bytes()` in Rust). The base seed is encoded as a 64-bit little-endian unsigned integer. The string literals `"ENSEMBLE_CHAIN_"` and `"_"` are encoded as ASCII bytes. The concatenation is therefore a fixed-structure 32-byte input with no ambiguity between chain indices or base-seed values. The resulting 256-bit SHA-256 hash is truncated to the least-significant 64 bits (bytes 0–7 of the output in little-endian order) as the `u64` output seed, which is then used to initialize the `SmallRng` state via `SeedableRng::seed_from_u64`.

4. Implementation

4.1. Crate Architecture

The REDIST-ENSEMBLE crate is structured as three source files within the `redist-ensemble` module of the `redist` Rust workspace:

spanning.rs Wilson's loop-erased random walk. Exports `wilson_ust(graph, rng)` returning an adjacency-list spanning tree. Uses `SmallRng` for per-step randomness (see Section 4.3).

recom.rs The ReCom proposal. Exports `recom_step(partition, graph, pop, tol, rng)`, which selects a pair, calls `wilson_ust`, enumerates balanced cuts, and returns the updated partition. Handles tree-resample-on-failure and pair-reselection logic.

chain.rs Parallel runner. Exports `run_ensemble(config)` which drives C independent chains via Rayon's `par_iter`, collecting per-step statistics and computing convergence diagnostics.

4.2. Graph Representation

The census-tract adjacency graph is stored as a `Vec<Vec<usize>>` adjacency list, loaded from the `.adj.bin` binary format shared with `redist-data`. Population data is a `Vec<u64>` aligned to the same vertex indices. Subgraphs for individual district pairs are constructed on demand as `Vec<Vec<usize>>` slices by remapping vertex indices to the compact range $\{0, \ldots, m-1\}$. This subgraph construction cost is $O(m)$ per step and does not allocate on the heap for graphs with $m \leq 512$ due to a stack-based small-buffer optimization.

4.3. Random Number Generation

Each chain uses a `SmallRng` instance from the `rand` crate, seeded from the SHA-256-derived per-chain seed. `SmallRng` is a 128-bit xoshiro128++ generator: its state is small (16 bytes), its throughput is high ($\sim$4 billion integers per second on modern hardware), and it passes the BigCrush statistical test suite. For the random walk within Wilson's algorithm, the dominant operation is selecting a uniformly random neighbor of the current vertex. We avoid modulo bias by using Lemire's fast divisor trick [Lemire, 2019] as implemented in `rand::Rng::gen_range`.

4.4. Balance Check

The balance check for a candidate spanning tree T of the merged region is implemented as a single depth-first traversal. We root T at vertex 0 of the local indexing and compute subtree population sums in $O(m)$ time using a post-order traversal. For each edge $(u, \mathit{parent}(u))$, the

subtree rooted at u has population $pop[u]$, and the complementary subtree has population $P_R - pop[u]$, where P_R is the total population of the merged region. The edge is *balanced* if both subtrees satisfy $|pop/\bar{p} - 1| \leq \varepsilon$. Balanced edges are collected into a `SmallVec<[usize; 8]>` to avoid heap allocation for the common case where the number of balanced cuts is small.

4.5. Serde Output

Step-level statistics—district population variances, normalized cut fraction, Democratic seat share (if election data is loaded)—are serialized to newline-delimited JSON using `serde_json`. Chain-level diagnostics ($\hat{R}$, ESS, Ham(1)) are written at chain completion. The output format is designed to be streamed directly into the `redist-report` pipeline for court-submission documents.

4.6. Texas and Bipartition Failures

Texas ($n = 4{,}866$ tracts, $k = 38$ districts) presents a known challenge: the state's shape and high k mean that a randomly merged pair of adjacent districts often contains a region with no balanced bipartition of any random spanning tree. GerryChain is documented to fail on Texas when run without a warmup from a known valid starting plan [MGGG Redistricting Lab, 2021].

redist-ensemble handles this via the pair-reselection mechanism in Algorithm 2: after 10 consecutive spanning tree resample failures for a given pair, a new adjacent pair is selected. In practice, for Texas, pair reselection triggers frequently during the first 50–100 steps from a cold start, then subsides as the chain moves away from the starting plan into regions of higher feasibility density. This behavior is not a Rust-specific limitation; it reflects the intrinsic geometry of the Texas plan space. Rust's advantage is that the 10-resample budget is exhausted in microseconds rather than the seconds it would take in Python, so pair reselection adds negligible overhead.

Stationarity under pair reselection. Pair reselection preserves the ReCom stationary distribution under mild conditions: when the pair-selection probability is symmetric and the pair is rejected only when no balanced cut exists (not based on the cut quality). In our implementation, pair reselection triggers only on balance failure — i.e., when all 10 sampled spanning trees for a given pair admit no balanced bipartition — matching GerryChain's

documented behavior. Under this protocol, the reselection step is equivalent to a proposal rejection that discards infeasible pair proposals uniformly, which preserves detailed balance if the pair-selection distribution is symmetric over adjacent district pairs. A formal ergodicity analysis is deferred to future work; the planned empirical G-track replication (Section 7) will compare ensemble distributions produced by REDIST-ENSEMBLE and GERRYCHAIN to confirm that any distributional difference is below statistical resolution.

5. Performance Analysis

5.1. GerryChain Baseline Measurements

We measured GERRYCHAIN throughput by running 1,000 RECOM steps for three states and recording wall-clock time. These are direct empirical measurements from the GERRYCHAIN Python package (version 0.3) on a standard development workstation:

State	k	n_{tracts}	1K steps (sec)	Steps/sec
North Carolina	14	2,672	47	≈ 21
Wisconsin	8	1,922	28	≈ 36
Georgia	14	1,978	30	≈ 33

The variation in throughput across states reflects differences in both n (tract count) and k (district count). Wisconsin is faster than North Carolina despite having fewer districts ($k = 8$ vs $k = 14$) because its lower tract count reduces the merged-region size; Georgia achieves similar throughput to Wisconsin despite $k = 14$ because its tracts are on average smaller in the adjacency graph.

5.2. Rust Target Estimates

The theoretical throughput of REDIST-ENSEMBLE follows from Wilson's complexity analysis. For a merged region of size $m = 2n/k$:

1. **Wilson's walk**: $O(m \log m)$ expected operations. At $m \approx 191$ (NC average: $2 \times 2{,}672/14$), Wilson requires approximately $191 \times \log(191) \approx 191 \times 7.5 \approx 1{,}433$ random walk steps.

2. **Effective operations per walk step**: Neighbor selection (1 uniform draw), loop detection (hash set lookup), and path extension (pointer write)—approximately 8 instructions at the clock level.

3. **Total operations per ReCom step**: $\approx 191 \times 8 = 1{,}528$ effective operations.

4. **Effective clock rate**: Modern CPUs execute at $\sim$4 GHz but MCMC-adjacent code (random number generation, irregular memory access) achieves roughly 1 GHz effective throughput on graph-walk workloads of this size.

5. **Resulting time per step**: $1{,}528 \,/\, 10^9 \approx 1.5\,\mu$s.

6. **Steps per second**: $\approx 660{,}000$.

Applying a conservative 13× overhead factor for Rust runtime costs (subgraph construction, adjacency list traversal, SmallVec allocation, serde bookkeeping) yields a practical target of **50,000 steps/sec**. This is still a 2,350× speedup over GerryChain for North Carolina.

Remark 1. *The overhead factor of* 13× *is deliberately conservative. Our METIS pipeline achieved a* 213× *speedup over Python for a structurally similar graph-partitioning workload [Della-Libera, 2026a]. The overhead factor for* redist-ensemble *is higher because Wilson's random walk has more irregular memory access patterns than METIS's multilevel contraction, but* 50,000 *steps/sec should be achievable without architecture-specific optimization.*

5.3. Projected Performance Table

5.4. Texas and California

GerryChain is documented to fail on Texas when started from a random plan: the bipartition feasibility rate for random district pairs in TX is low enough that the chain stalls indefinitely without a carefully constructed warm start. redist-ensemble's pair-reselection mechanism (Section 4) enables it to traverse the low-feasibility region rapidly. The performance estimate in Table 1 assumes the pair reselection overhead is absorbed within the 50,000 steps/sec budget.

Per-step cost is $O((n/k)\log(n/k))$, so the relevant quantity is the average merged-region size $m \approx 2n/k$. For CA ($k = 52$, $n = 8{,}057$): $m \approx 310$. For PA ($k = 17$, $n = 3{,}218$): $m \approx 379$.

Table 1: Performance comparison: GerryChain measured vs. redist-ensemble estimated. Rust estimates based on Wilson's $O((n/k)\log(n/k))$ analysis at 50,000 steps/sec (conservative). PA and TX estimated from n-scaling of the NC/WI/GA measurements. All Rust figures require validation against `criterion.rs` (Phase 2).

State	k	n	GerryChain (measured)		redist-ensemble (estimated†)	
			1K steps (s)	Steps/s	10K steps (s)	Speedup
NC	14	2,672	47	21	0.20	2,350×
WI	8	1,922	28	36	0.12	2,330×
GA	14	1,978	30	33	0.13	2,310×
PA	17	3,218	∼30†	∼33	0.18	2,200×
TX	38	4,866	failure‡	—	0.31	—‡
CA	52	8,057	failure‡	—	0.52	—‡

† Rust estimates require validation with `criterion.rs`; see Section 5.5.

‡ GerryChain *without* pair reselection fails on TX ($k = 38$) and CA ($k = 52$) from cold start due to bipartition infeasibility; GerryChain *with* pair reselection and redist-ensemble both handle these cases via pair reselection (Section 5.4). GerryChain's bipartition failures on TX $k = 38$ are combinatorial, not language-specific. Both GerryChain-with-pair-reselection and redist-ensemble succeed. The advantage of redist-ensemble is throughput, not correctness.

PA has the largest average merged region among the six G-track states and therefore the highest per-step cost, not California. The estimated 0.52 seconds for 10,000 CA steps and 0.18 seconds for 10,000 PA steps (Table 1) reflect this relationship. Both represent dramatic improvements over GerryChain's inability to complete even 1,000 steps from a cold start for TX and CA.

5.5. Phase 2: Criterion.rs Benchmarks

All Rust throughput figures in this paper are estimates derived from complexity analysis. Phase 2 of the redist-ensemble project will validate these estimates using `criterion.rs`, Rust's standard micro-benchmarking harness. Benchmarks will be run for each of the six G-track states (NC, WI, GA, PA, TX, CA) at the following granularities:

- **Wilson UST**: Time to generate one uniform random spanning tree for a merged region of size m, as a function of m.
- **Balance check**: Time for the full edge enumeration pass.
- **Full step**: End-to-end ReCom step time including subgraph construction.

- **Chain throughput**: Steps/sec for a 1,000-step chain, compared directly against GerryChain wall-clock time.

Results will be reported in a companion note to this paper.

6. Convergence Diagnostics

The redist-ensemble chain runner computes three convergence diagnostics in the same pass as the sampling, at zero additional per-step cost beyond tracking per-chain statistics. The diagnostic framework is described in full in Della-Libera [2026d]; we summarize the implementations here for completeness.

6.1. R-hat (Potential Scale Reduction Factor)

The Gelman-Rubin $\hat{R}$ statistic [Gelman and Rubin, 1992] compares between-chain variance to within-chain variance for a scalar summary statistic θ (e.g., Democratic seat count or Polsby-Popper compactness). For C chains each of length N, let $\bar{\theta}_{c\cdot}$ denote the within-chain mean and $\bar{\theta}_{\cdot\cdot}$ the grand mean. The between-chain variance is

$$B = \frac{N}{C-1} \sum_{c=1}^{C} (\bar{\theta}_{c\cdot} - \bar{\theta}_{\cdot\cdot})^2,$$

and the within-chain variance is

$$W = \frac{1}{C} \sum_{c=1}^{C} s_c^2, \quad s_c^2 = \frac{1}{N-1} \sum_{t=1}^{N} (\theta_{ct} - \bar{\theta}_{c\cdot})^2.$$

The $\hat{R}$ statistic is

$$\hat{R} = \sqrt{\frac{(N-1)/N \cdot W + B/N}{W}}.$$

Values of $\hat{R} < 1.1$ indicate approximate convergence; we target $\hat{R} < 1.05$ for publication-quality results.

For redistricting statistics, which are discrete (seat counts take integer values) or bounded (compactness metrics lie in $[0, 1]$), we follow Vehtari et al. [2021] and apply the rank-normalised $\hat{R}$ variant. All CN draws across all chains are first rank-transformed to approximate normality, then $\hat{R}$ is computed on the transformed values. Rank normalisation removes sensitivity to the

marginal distribution shape and is the current best practice for MCMC diagnostics [Vehtari et al., 2021].

Implementation. Each chain accumulates $\bar{\theta}_{c\cdot}$ and s_c^2 incrementally using Welford's online algorithm [Welford, 1962], requiring $O(1)$ storage per chain. For rank normalisation, all step-level draws are buffered in a `Vec<f64>` and rank-sorted at chain completion. $\hat{R}$ is computed in a single pass over the C chain statistics.

6.2. Effective Sample Size

The effective sample size accounts for autocorrelation within each chain. Using Geyer's initial monotone sequence estimator [Geyer, 1992], the ESS for a single chain of length N is

$$\text{ESS} = \frac{N}{1 + 2\sum_{k=1}^{\hat{K}} \hat{\rho}_k},$$

where $\hat{\rho}_k$ is the lag-k autocorrelation estimate and $\hat{K}$ is the cutoff where the monotone sequence estimate becomes non-positive. For redistricting chains, typical lag-1 autocorrelation is $\hat{\rho}_1 \approx 0.85$–$0.95$, giving ESS $\approx$ 500–2,000 for a 10,000-step chain.

We use the multi-chain ESS formula from Vehtari et al. [2021], which pools within-chain and between-chain information:

$$\text{ESS}_{\text{bulk}} = \frac{CN \cdot \min(\hat{V}/B, 1)}{1 + 2\sum_{k=1}^{\hat{K}} \hat{\rho}_k^{+}},$$

where $\hat{V} = W + B/N$ and $\hat{\rho}_k^{+}$ is the rank-normalised autocorrelation at lag k.

Target. We require ESS ≥ 400 for any summary statistic used in a legal filing. For a 10,000-step, 8-chain run at typical redistricting autocorrelation, the expected ESS is 4,000–16,000—well above this threshold.

6.3. Hamming Autocorrelation

The Hamming distance between two redistricting plans σ and σ' is the fraction of census tracts assigned to different districts:

$$d_H(\sigma, \sigma') = \frac{|\{v : \sigma(v) \neq \sigma'(v)\}|}{n}.$$

The lag-k Hamming autocorrelation is defined as

$$\mathrm{Ham}(k) = \mathrm{Corr}(d_H(\sigma_t, \sigma_0), d_H(\sigma_{t+k}, \sigma_0)),$$

where the correlation is taken across steps t. In practice, $\hat{\rho}_1 \approx 0.87$–$0.93$ for ReCom, with geometric decay: $\mathrm{Ham}(k) \approx \hat{\rho}_1^k$.

The Hamming autocorrelation serves as a distribution-free mixing diagnostic: it measures how rapidly consecutive plans decorrelate in plan space, independent of any particular summary statistic. A chain with $\mathrm{Ham}(1) > 0.98$ is mixing slowly and may require thinning or a longer run; $\mathrm{Ham}(1) < 0.90$ indicates healthy mixing.

The normalized cut fraction $\phi(\sigma) = |E_\sigma|/|E|$ (the fraction of edges crossing district boundaries) is used as a computationally efficient proxy for plan identity when computing Ham(1). This proxy is exact in the sense that ϕ is the same summary statistic minimized by METIS and reported by the ApportionRegions evaluation.

6.4. Convergence Targets for Publication

Table 2 summarizes the diagnostic thresholds. The target is to achieve all three thresholds after 5,000 steps for all six G-track states. At redist-ensemble's estimated throughput of 50,000 steps/sec, 5,000 steps require approximately 0.1 seconds.

Table 2: Convergence diagnostic thresholds for redist-ensemble publication results.

Diagnostic	Symbol	Target
Gelman-Rubin (rank-normalised)	$\hat{R}$	< 1.05
Effective sample size	ESS	≥ 400 per statistic
Hamming autocorrelation	Ham(1)	0.85–0.95

7. Conclusion

7.1. Summary

We have presented REDIST-ENSEMBLE, a Rust implementation of the RECOM Markov chain for redistricting ensemble analysis. The core contribution is the use of Wilson's uniform random spanning tree algorithm, which provides $O((n/k)\log(n/k))$ expected per-step cost, combined with Rust's compiled execution to achieve an estimated 2,300× throughput improvement over GERRYCHAIN.

The practical consequence is a shift in the role of ensemble analysis within the redistricting pipeline. At GERRYCHAIN speeds, a 10,000-step ensemble for North Carolina requires roughly 8 minutes; at REDIST-ENSEMBLE's target throughput, the same ensemble requires approximately 0.2 seconds. This puts ensemble analysis in the same computational tier as compactness scoring and demographic analysis—a routine per-state audit step rather than an overnight computation.

The implementation addresses three correctness requirements that distinguish REDIST-ENSEMBLE from a naive Rust port: (1) full balanced-cut enumeration to match GerryChain's stationary distribution; (2) tree-resample-on-failure rather than same-tree retry; (3) pair-reselection after 10 consecutive tree failures. The TX and CA cold-start challenge is combinatorial, not language-specific: GERRYCHAIN with pair reselection also handles these states. REDIST-ENSEMBLE's advantage is that Rust's throughput reduces the wall time for the pair-reselection warm-up phase from seconds (Python) to microseconds, making cold-start runs practical within a pipeline budget. Convergence diagnostics ($\hat{R}$, ESS, Ham(1)) are computed in-pass using Welford's online algorithm and Geyer's initial monotone sequence estimator, adding no asymptotic overhead.

7.2. Methodological Significance

REDIST-ENSEMBLE closes the final methodological gap in the G-track research series. The G-track papers (G.0–G.5) establish the theoretical and empirical framework for placing APPORTIONREGIONS plans within RECOM ensembles, but their computational results previously depended on a Python GERRYCHAIN installation. REDIST-ENSEMBLE makes those results reproducible from the `redist` Rust binary alone, satisfying the AEA replication

standard for the full portfolio.

7.3. Future Work

Three directions remain for Phase 2:

Benchmark validation. All throughput figures in this paper are complexity-analysis estimates. `criterion.rs` benchmarks are required to validate the 50,000 steps/sec target and characterize the overhead breakdown (subgraph construction, Wilson walk, balance enumeration, serde output).

Pipeline integration. The `redist ensemble` subcommand should be integrated into `redist build` as an automatic post-redistricting audit step, running 10,000 steps from the produced plan and reporting $\hat{R}$, ESS, Ham(1), and the plan's percentile rank in the resulting ensemble distribution.

Empirical G-track replication. Once throughput is validated, the six G-track states (NC, WI, GA, PA, TX, CA) should be re-run using REDIST-ENSEMBLE in place of GERRYCHAIN, and the resulting ensemble distributions compared against the G.1 results to confirm that the Rust and Python implementations produce statistically indistinguishable output. Agreement would certify that REDIST-ENSEMBLE's algorithmic choices correctly replicate GERRYCHAIN's stationary distribution.

References

David Aldous. The random walk construction of uniform spanning trees and uniform labelled trees. *SIAM Journal on Discrete Mathematics*, 3(4):450–465, 1990. doi: 10.1137/0403039.

Eric A. Autry, Daniel Carter, Gregory J. Herschlag, Zach Hunter, and Jonathan C. Mattingly. Metropolized multiscale forest recombination for redistricting. *Multiscale Modeling & Simulation*, 19(4):1885–1914, 2021. doi: 10.1137/21M1406854.

Daniel Carter, Zach Hunter, Gregory Herschlag, and Jonathan Mattingly. A merge-split proposal for reversible Monte Carlo Markov chain sampling of redistricting plans. *arXiv preprint arXiv:1911.01503*, 2019.

Maria Chikina, Alan Frieze, and Wesley Pegden. Assessing significance in a Markov chain without mixing. *Proceedings of the National Academy of Sciences*, 114(11):2860–2864, 2017. doi: 10.1073/pnas.1617096114.

Daryl DeFord, Moon Duchin, and Justin Solomon. Recombination: A family of markov chains for redistricting. *Harvard Data Science Review*, 3(1), 2021. doi: 10.1162/99608f92.eb30390f.

Gio Della-Libera. From apportionment to boundary design: Extending huntington-hill to congressional redistricting. *Working paper*, 2026a. B.1 — foundational recursive bisection paper.

Gio Della-Libera. Algorithmic redistricting and ensemble methods: A framework for comparison. *Working paper*, 2026b. G.0 — G-track methodology framework.

Gio Della-Libera. Where does the algorithmic map fall? ApportionRegions vs. GerryChain ensemble for six states. *Working paper*, 2026c. G.1 — direct percentile placement.

Gio Della-Libera. Convergence diagnostics for redistricting Markov chains: $\hat{R}$, ESS, and Hamming autocorrelation. *Working paper*, 2026d. G.4 — ensemble diagnostics formalisation.

Gio Della-Libera. Bisection-ensemble duality in algorithmic redistricting. *Working paper*, 2026e. H.1 — bisection-ensemble connection.

Andrew Gelman and Donald B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472, 1992. doi: 10.1214/ss/1177011136.

Charles J. Geyer. Practical Markov chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992. doi: 10.1214/ss/1177011137.

Gregory Herschlag, Han Sung Kang, Justin Luo, Christy Vaughn Graves, Sachet Bangia, Robert Ravier, and Jonathan C. Mattingly. Quantifying gerrymandering in north carolina. *Statistics and Public Policy*, 7(1):30–38, 2020. doi: 10.1080/2330443X.2020.1828567.

Gregory F. Lawler. A self-avoiding random walk. *Duke Mathematical Journal*, 47(3):655–693, 1980. doi: 10.1215/S0012-7094-80-04741-9.

Daniel Lemire. Fast random integer generation in an interval, 2019.

Nicholas D. Matsakis and Felix S. Klock. The Rust language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology*, pages 103–104, 2014. doi: 10.1145/2663171.2663188.

MGGG Redistricting Lab. GerryChain: A python package for redistricting ensembles. `https://github.com/mggg/GerryChain`, 2021. Version 0.3.

James G. Propp and David B. Wilson. How to get a perfectly random sample from a generic Markov chain and generate a random spanning tree of a directed graph. *Journal of Algorithms*, 27(2):170–217, 1998. doi: 10.1006/jagm.1997.0917.

Rayon Contributors. Rayon: A data parallelism library for Rust. `https://github.com/rayon-rs/rayon`, 2024.

Supreme Court of the United States. *Rucho v. Common Cause*, 588 u.s. 684 (2019), 2019.

Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2):667–718, 2021. doi: 10.1214/20-BA1221.

B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962. doi: 10.1080/00401706.1962.10490022.

David B. Wilson. Generating random spanning trees more quickly than the cover time. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 296–303. ACM, 1996. doi: 10.1145/237814.237880.